

Machine Learning Algorithms Laboratory

Experiment 4

Binary Classification using Linear and Kernel-Based Models

Name: Jayasri M
Register No.: 3122247001025

Aim and Objective

To classify emails as spam or ham using Logistic Regression and Support Vector Machine (SVM) and compare their performance using different evaluation metrics.

Dataset Description

The Spambase dataset was used in this experiment. It contains numerical features extracted from email messages and a target column that classifies emails as spam or ham.

Preprocessing Steps

The following preprocessing steps were carried out.

- Loaded the dataset.
- Checked the dataset information.
- Verified missing values.
- Standardized the input features.
- Performed exploratory data analysis.
- Split the dataset into training and testing sets.

Implementation Details

The following models were implemented.

- Logistic Regression
- Support Vector Machine (SVM)

GridSearchCV was used to obtain suitable hyperparameters for both classifiers. Fivefold cross validation was performed to evaluate the models.

```
import time

import pandas as pd
import matplotlib.pyplot as plt
```

```

from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import ConfusionMatrixDisplay
from google.colab import files
uploaded = files.upload()
<IPython.core.display.HTML
object>
Saving      spambase_csv.csv      to
spambase_csv (1).csv
data = pd.read_csv("spambase_csv.csv")
data.head()

{"type":"dataframe","variable_name":"data"}

print("Shape :", data.shape)

print("\nColumns")
print(data.columns)

print("\nMissing Values")
print(data.isnull().sum())
data.info()
Shape : (4601,
58)

Columns
Index(['word_freq_make', 'word_freq_address', 'word_freq_all',
'word_freq_3d',
      'word_freq_our', 'word_freq_over', 'word_freq_remove',
      'word_freq_internet', 'word_freq_order', 'word_freq_mail',
      'word_freq_receive', 'word_freq_will', 'word_freq_people',
      'word_freq_report', 'word_freq_addresses', 'word_freq_free',
      'word_freq_business', 'word_freq_email', 'word_freq_you',
      'word_freq_credit', 'word_freq_your', 'word_freq_font',
      'word_freq_000',

```

```

        'word_freq_money', 'word_freq_hp', 'word_freq_hpl',
'word_freq_george',
        'word_freq_650', 'word_freq_lab', 'word_freq_labs',
'word_freq_telnet',
        'word_freq_857', 'word_freq_data', 'word_freq_415',
'word_freq_85',
        'word_freq_technology', 'word_freq_1999', 'word_freq_parts',
        'word_freq_pm', 'word_freq_direct', 'word_freq_cs',
'word_freq_meeting',
        'word_freq_original', 'word_freq_project', 'word_freq_re',
        'word_freq_edu', 'word_freq_table', 'word_freq_conference',
        'char_freq_%3B', 'char_freq_%28', 'char_freq_%5B', 'char_freq_
%21',
        'char_freq_%24', 'char_freq_%23', 'capital_run_length_average',
        'capital_run_length_longest', 'capital_run_length_total',
'class'],
dtype='object')

```

Missing Values

word_freq_make	0	
word_freq_address	0	
word_freq_all	0	word_freq_3d
0 word_freq_our	0	
word_freq_over	0	
word_freq_remove	0	
word_freq_internet	0	
word_freq_order	0	
word_freq_mail	0	
word_freq_receive	0	
word_freq_will	0	
word_freq_people	0	
word_freq_report	0	
word_freq_addresses	0	
word_freq_free	0	
word_freq_business	0	
word_freq_email	0	
word_freq_you	0	
word_freq_credit	0	
word_freq_your	0	
word_freq_font	0	
word_freq_000	0	
word_freq_money	0	word_freq_hp
0 word_freq_hpl	0	

```

word_freq_george      0
word_freq_650         0
word_freq_lab         0
word_freq_labs        0
word_freq_telnet      0
word_freq_857         0
word_freq_data        0
word_freq_415         0
word_freq_85          0
word_freq_technology  0
word_freq_1999        0
word_freq_parts       0
word_freq_pm          0
word_freq_direct      0
word_freq_cs          0
word_freq_meeting     0
word_freq_original    0
word_freq_project     0
word_freq_re          0
word_freq_edu         0
word_freq_table       0
word_freq_conference  0
char_freq_%3B         0
char_freq_%28         0
char_freq_%5B         0
char_freq_%21         0
char_freq_%24         0
char_freq_%23         0
capital_run_length_average  0
capital_run_length_longest  0
capital_run_length_total  0
class                 0

```

```
dtype: int64
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 4601 entries, 0 to 4600 Data
```

```
columns (total 58 columns):
```

#	Column	Non-Null Count	Dtype
0	word_freq_make	4601 non-null	float64
1	word_freq_address	4601 non-null	float64
2	word_freq_all	4601 non-null	float64
3	word_freq_3d	4601 non-null	float64
4	word_freq_our	4601 non-null	float64
5	word_freq_over	4601 non-null	float64
6	word_freq_remove	4601 non-null	float64
7	word_freq_internet	4601 non-null	float64
8	word_freq_order	4601 non-null	float64
9	word_freq_mail	4601 non-null	float64

10	word_freq_receive	4601 non-null	float64
11	word_freq_will	4601 non-null	float64
12	word_freq_people	4601 non-null	float64
13	word_freq_report	4601 non-null	float64
14	word_freq_addresses	4601 non-null	float64
15	word_freq_free	4601 non-null	float64
16	word_freq_business	4601 non-null	float64
17	word_freq_email	4601 non-null	float64
18	word_freq_you	4601 non-null	float64
19	word_freq_credit	4601 non-null	float64
20	word_freq_your	4601 non-null	float64
21	word_freq_font	4601 non-null	float64
22	word_freq_000	4601 non-null	float64
23	word_freq_money	4601 non-null	float64
24	word_freq_hp	4601 non-null	float64
25	word_freq_hpl	4601 non-null	float64
26	word_freq_george	4601 non-null	float64
27	word_freq_650	4601 non-null	float64
28	word_freq_lab	4601 non-null	float64
29	word_freq_labs	4601 non-null	float64
30	word_freq_telnet	4601 non-null	float64
31	word_freq_857	4601 non-null	float64
32	word_freq_data	4601 non-null	float64
33	word_freq_415	4601 non-null	float64
34	word_freq_85	4601 non-null	float64
35	word_freq_technology	4601 non-null	float64
36	word_freq_1999	4601 non-null	float64
37	word_freq_parts	4601 non-null	float64
38	word_freq_pm	4601 non-null	float64
39	word_freq_direct	4601 non-null	float64
40	word_freq_cs	4601 non-null	float64
41	word_freq_meeting	4601 non-null	float64
42	word_freq_original	4601 non-null	float64
43	word_freq_project	4601 non-null	float64
44	word_freq_re	4601 non-null	float64
45	word_freq_edu	4601 non-null	float64
46	word_freq_table	4601 non-null	float64
47	word_freq_conference	4601 non-null	float64
48	char_freq_%3B	4601 non-null	float64
49	char_freq_%28	4601 non-null	float64
50	char_freq_%5B	4601 non-null	float64
51	char_freq_%21	4601 non-null	float64
52	char_freq_%24	4601 non-null	float64
53	char_freq_%23	4601 non-null	float64
54	capital_run_length_average	4601 non-null	float64
55	capital_run_length_longest	4601 non-null	int64

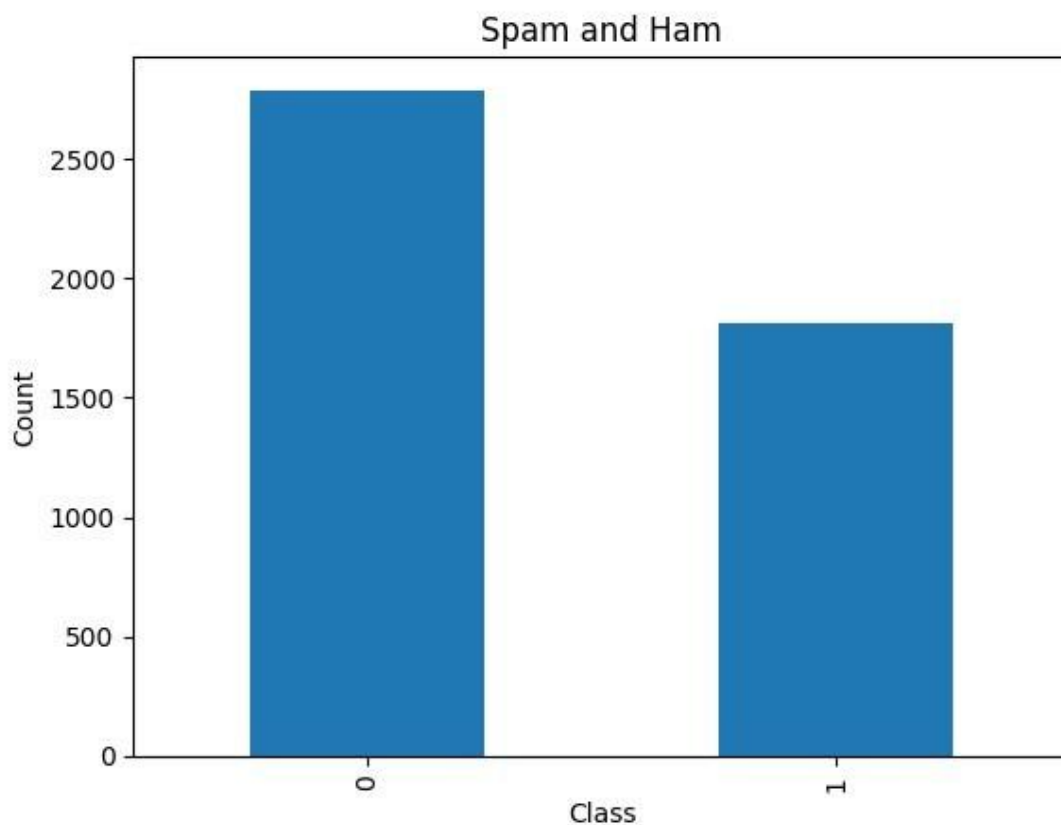
```
56 capital_run_length_total    4601 non-null    int64
57 class                        4601 non-null    int64
dtypes: float64(55), int64(3)
memory usage: 2.0 MB
```

```
X = data.drop("class", axis=1)

y = data["class"] scaler =
StandardScaler()

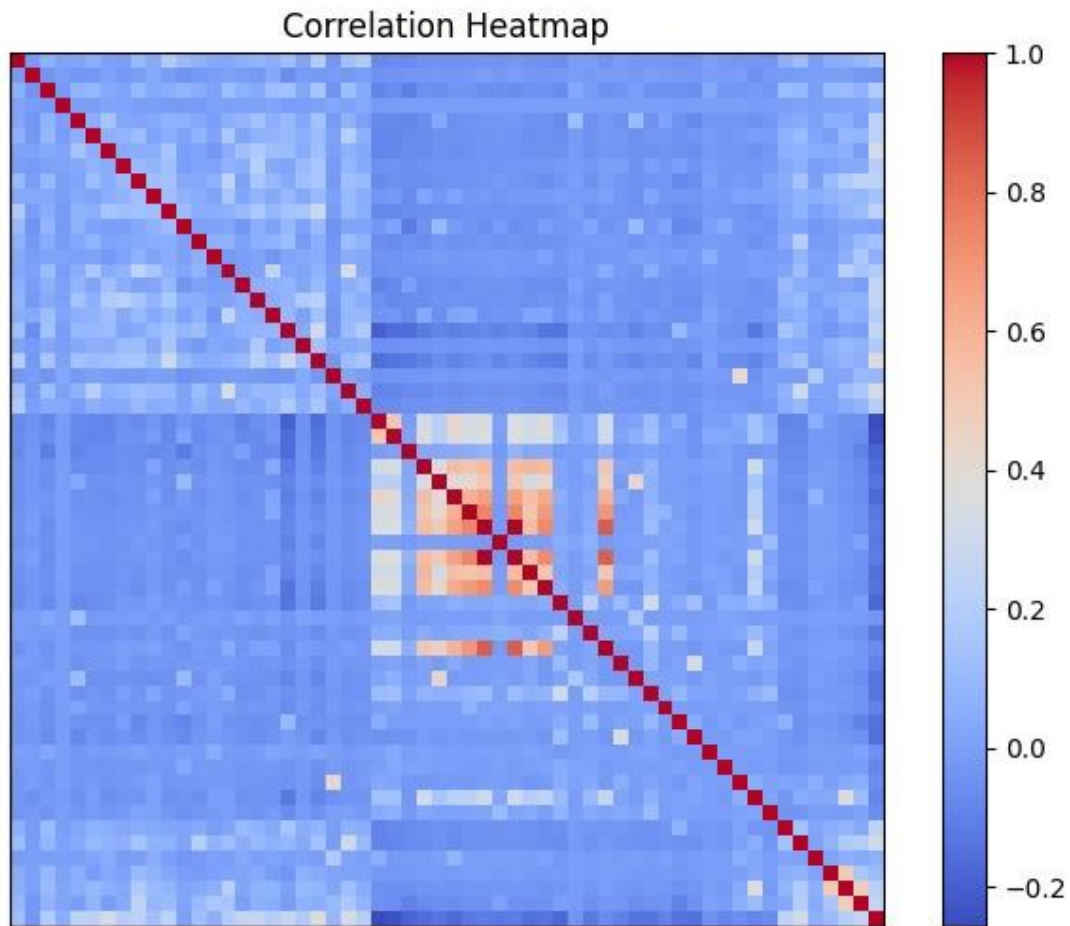
X = scaler.fit_transform(X)

data["class"].value_counts().plot(kind="bar")
plt.title("Spam and Ham") plt.xlabel("Class")
plt.ylabel("Count") plt.show()
```



```
plt.figure(figsize=(8,6)) plt.imshow(data.corr(),  
cmap="coolwarm") plt.colorbar()
```

```
plt.title("Correlation Heatmap")
plt.xticks([]) plt.yticks([])
plt.show()
```



```
X_train, X_test, y_train, y_test =
train_test_split(X, y,
                 test_size=0.2,
                 random_state=42
                 )

print("Training :", X_train.shape)
print("Testing :", X_test.shape)
```

```

Training : (3680, 57)
Testing : (921, 57)

logistic = LogisticRegression(max_iter=1000) start
= time.time()

logistic.fit(X_train, y_train)

logistic_time = time.time() - start

logistic_pred = logistic.predict(X_test)

accuracy = accuracy_score(y_test, logistic_pred)
precision = precision_score(y_test, logistic_pred)
recall = recall_score(y_test, logistic_pred) f1 =
f1_score(y_test, logistic_pred)

result = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1 Score", "Training
Time"],
    "Value": [accuracy, precision, recall, f1, logistic_time]
})

result

{"summary": "{\n  \"name\": \"result\",\n  \"rows\": 5,\n  \"fields\":
[\n    {\n      \"column\": \"Metric\",\n      \"properties\": {\n
\"dtype\": \"string\",\n      \"num_unique_values\": 5,\n
\"samples\": [\n        \"Precision\",\n        \"Training
Time\",\n        \"Recall\",\n        ],\n        \"semantic_type\":
\"\", \n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"Value\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.3917353112178201,\n        \"min\":
0.03233623504638672,\n        \"max\": 0.9316939890710383,\n
\"num_unique_values\": 5,\n        \"samples\": [\n
0.9316939890710383,\n        0.03233623504638672,\n
0.8743589743589744\n        ],\n        \"semantic_type\": \"\", \n
\"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\",
\"variable_name\": \"result\"}

parameters = {
    "C": [0.1, 1, 10],
    "solver": ["liblinear"]
}

grid = GridSearchCV(
    LogisticRegression(max_iter=5000),
    parameters,
    cv=5,
    scoring="accuracy",

```

```

        n_jobs=-1
    )

    grid.fit(X_train, y_train)

    print("Best Parameters :", grid.best_params_) print("Best
    Accuracy :", grid.best_score_)
    Best Parameters : {'C': 10, 'solver': 'liblinear'}
    Best Accuracy : 0.9258152173913043

    linear_model = SVC(kernel="linear")

    start = time.time()

    linear_model.fit(X_train, y_train) linear_time
    = time.time() - start

    linear_prediction = linear_model.predict(X_test)

    poly_model =
    SVC(      kernel="poly",
    degree=3
    )

    start = time.time()

    poly_model.fit(X_train, y_train)

    poly_time = time.time() - start

    poly_prediction = poly_model.predict(X_test)

    rbf_model = SVC(kernel="rbf") start =
    time.time()

    rbf_model.fit(X_train, y_train)

    rbf_time = time.time() - start

    rbf_prediction = rbf_model.predict(X_test)

    sigmoid_model = SVC(kernel="sigmoid")

    start = time.time()

    sigmoid_model.fit(X_train, y_train) sigmoid_time
    = time.time() - start

```

```

sigmoid_prediction = sigmoid_model.predict(X_test)

svm_parameters = {
    "kernel": ["linear", "rbf"],
    "C": [1, 10],
    "gamma": ["scale"]
}

svm_grid =
GridSearchCV(      SVC(),
svm_parameters,
cv=5,
    scoring="accuracy",
n_jobs=-1
)

svm_grid.fit(X_train, y_train)

print("Best Parameters :", svm_grid.best_params_) print("Best
Accuracy :", svm_grid.best_score_)
Best Parameters : {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
Best Accuracy : 0.93125

logistic_result = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score",
        "Training Time"
    ],
    "Value": [
        accuracy_score(y_test, logistic_pred),
precision_score(y_test, logistic_pred),
recall_score(y_test, logistic_pred),          f1_score(y_test,
logistic_pred),
        logistic_time
    ]
})

logistic_result

{"summary": "{\n  \"name\": \"logistic_result\",\n  \"rows\": 5,\n  \"fields\": [\n    {\n      \"column\": \"Metric\",\n  \"properties\": {\n      \"dtype\": \"string\",\n  \"num_unique_values\": 5,\n  \"samples\": [\n  \"Precision\",\n      \"Training Time\",\n      \"Recall\"\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n    },\n    {\n      \"column\": \"Value\",\n  \"properties\":
```

```

{\n          \"dtype\": \"number\", \n          \"std\":
0.3917353112178201, \n          \"min\": 0.03233623504638672, \n
\"max\": 0.9316939890710383, \n          \"num_unique_values\": 5, \n
\"samples\": [\n          0.9316939890710383, \n
0.03233623504638672, \n          0.8743589743589744 \n          ], \n
\"semantic_type\": \"\", \n          \"description\": \"\" \n          } \n
    ] \n }\", \"type\": \"dataframe\", \"variable_name\": \"logistic_result\"}

svm_result = pd.DataFrame({
    \"Kernel\": [
        \"Linear\",
        \"Polynomial\",
        \"RBF\",
        \"Sigmoid\"
    ],
    \"Accuracy\": [
        accuracy_score(y_test, linear_prediction),
        accuracy_score(y_test, poly_prediction),
        accuracy_score(y_test, rbf_prediction),
        accuracy_score(y_test,
        sigmoid_prediction)
    ],
    \"F1 Score\": [
        f1_score(y_test, linear_prediction),
        f1_score(y_test, poly_prediction),
        f1_score(y_test, rbf_prediction),
        f1_score(y_test,
        sigmoid_prediction)
    ],
    \"Training Time\":
    [
        linear_time,
        poly_time,
        rbf_time,
        sigmoid_time
    ]
})

svm_result

{\"summary\": \"{ \n  \"name\": \"svm_result\", \n  \"rows\": 4, \n
\"fields\": [ \n    { \n      \"column\": \"Kernel\", \n
\"properties\": { \n        \"dtype\": \"string\", \n
\"num_unique_values\": 4, \n        \"samples\": [ \n
\"Polynomial\", \n        \"Sigmoid\", \n        \"Linear\" \n
], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n
} \n    }, \n    { \n      \"column\": \"Accuracy\", \n
\"properties\": { \n        \"dtype\": \"number\", \n        \"std\":
0.07849194168046825, \n        \"min\": 0.7643865363735071, \n
\"max\": 0.9348534201954397, \n        \"num_unique_values\": 4, \n
\"samples\": [ \n        0.7643865363735071, \n
0.8892508143322475, \n        0.9250814332247557 \n        ], \n
\"semantic_type\": \"\", \n        \"description\": \"\" \n
} \n

```

```

n      },\n      {\n          \"column\": \"F1 Score\", \n          \"properties\": {\n              \"dtype\": \"number\", \n              \"std\": 0.13683533412173748, \n              \"min\": 0.629059829059829, \n              \"max\": 0.9206349206349206, \n              \"num_unique_values\": 4, \n              \"samples\": [\n                  0.629059829059829, \n                  0.8664921465968587, \n                  0.9090909090909091\n              ], \n              \"semantic_type\": \"\", \n              \"description\": \"\"\n          }\n      },\n      {\n          \"column\": \"Training Time\", \n          \"properties\": {\n              \"dtype\": \"number\", \n              \"std\": 0.18439156291989828, \n              \"min\": 0.3518345355987549, \n              \"max\": 0.7563803195953369, \n              \"num_unique_values\": 4, \n              \"samples\": [\n                  0.6066489219665527, \n                  0.4152810573577881, \n                  0.7563803195953369\n              ], \n              \"semantic_type\": \"\", \n              \"description\": \"\"\n          }\n      }\n  ],\n  \"type\": \"dataframe\", \"variable_name\": \"svm_result\"}

```

```

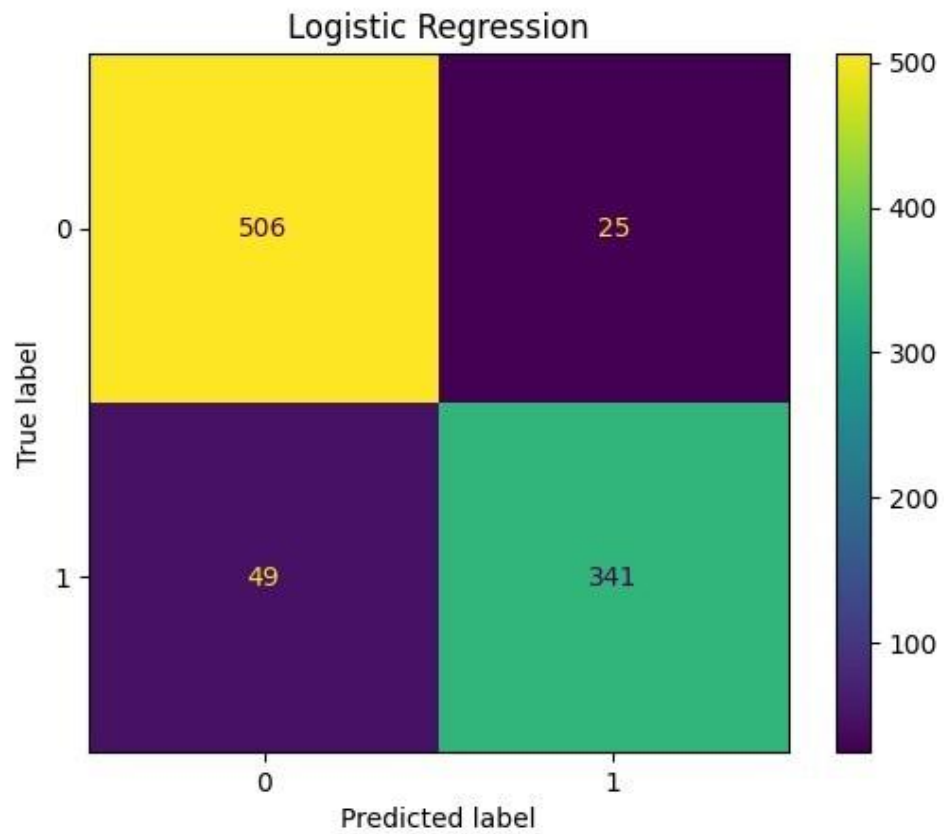
ConfusionMatrixDisplay.from_predictions(
    y_test,
    logistic_pred )

```

```

plt.title("Logistic Regression")
plt.show()

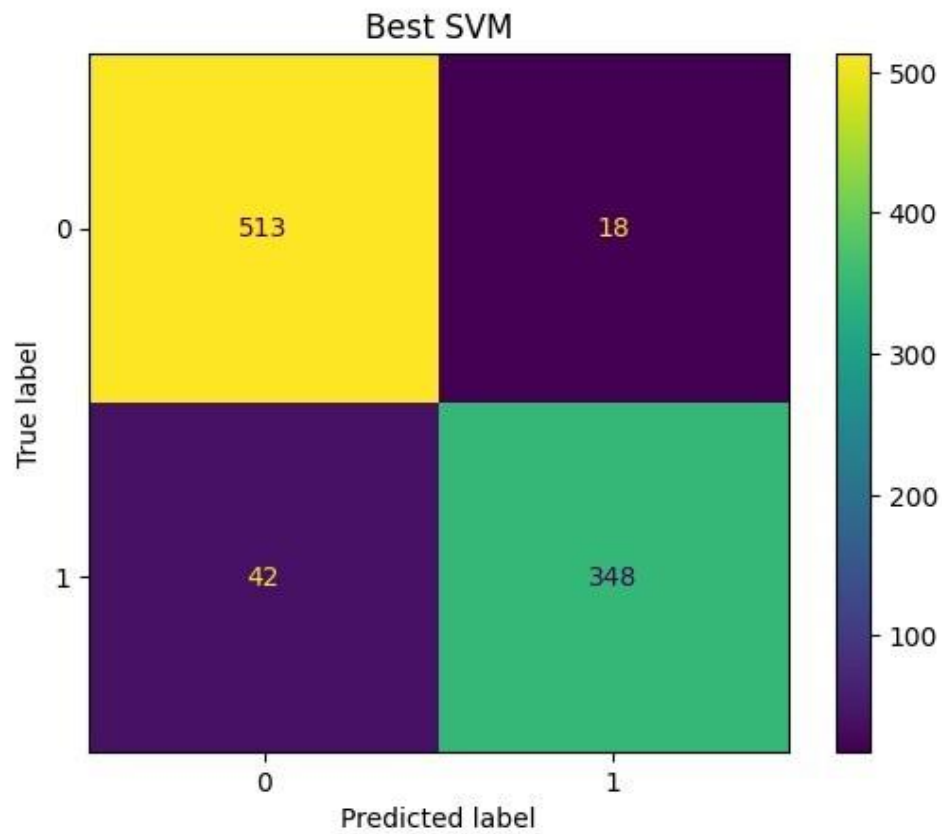
```



```
best_model = svm_grid.best_estimator_
best_prediction = best_model.predict(X_test)

ConfusionMatrixDisplay.from_predictions(
    y_test,
    best_prediction )

plt.title("Best SVM")
plt.show()
```



```
logistic_cv = cross_val_score(  
    logistic,  
    X,  
    y,  
    cv=5,  
    scoring="accuracy"  
)  
  
svm_cv =  
cross_val_score(    best_model,  
    X,  
    y,  
    cv=5,  
    scoring="accuracy"  
)  
  
cv_result = pd.DataFrame({  
    "Fold": [  
        "Fold 1",  
        "Fold 2",  
        "Fold 3",  
        "Fold 4",  
        "Fold 5",  
    ],  
    "logistic_cv": logistic_cv,  
    "svm_cv": svm_cv,  
})
```

```

        "Average"
    ],
    "Logistic Regression":
    [
        logistic_cv[0],
        logistic_cv[1],
        logistic_cv[2],
        logistic_cv[3],
        logistic_cv[4],
        logistic_cv.mean()
    ],
    "SVM":
    [
        svm_cv[0],
        svm_cv[1],
        svm_cv[2],
        svm_cv[3],
        svm_cv[4],
        svm_cv.mean()
    ]
})

cv_result

{"summary": "{\n  \"name\": \"cv_result\",\n  \"rows\": 6,\n  \"fields\": [\n    {\n      \"column\": \"Fold\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 6,\n        \"samples\": [\n          \"Fold 1\",\n          \"Fold 2\",\n          \"Average\"],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Logistic Regression\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.0362081523775666,\n        \"min\": 0.8402173913043478,\n        \"max\": 0.9402173913043478,\n        \"num_unique_values\": 6,\n        \"samples\": [\n          0.9185667752442996,\n          0.9282608695652174,\n          0.9113220507010338],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"SVM\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.037249441529529664,\n        \"min\": 0.85,\n        \"max\": 0.95,\n        \"num_unique_values\": 6,\n        \"samples\": [\n          0.9326818675352877,\n          0.933695652173913,\n          0.9230581126374922],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}", "type": "dataframe", "variable_name": "cv_result"}

tuning_result = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Support Vector Machine"
    ],
    "Search Method": [
        "GridSearchCV",

```

```

        "GridSearchCV"
    ],
    "Best Parameters":
    [
        str(grid.best_params_),
        str(svm_grid.best_params_)
    ],
    "Best CV Accuracy":
    [
        grid.best_score_,
        svm_grid.best_score_
    ]
})

tuning_result

{"summary": "{\n  \"name\": \"tuning_result\",\n  \"rows\": 2,\n  \"fields\": [\n    {\n      \"column\": \"Model\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"Support Vector Machine\",\n          \"Logistic Regression\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Search Method\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 1,\n        \"samples\": [\n          \"GridSearchCV\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Best Parameters\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"{ 'C': 1, 'gamma': 'scale', 'kernel': 'rbf' }\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Best CV Accuracy\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.0038429716368834277,\n        \"min\": 0.9258152173913043,\n        \"max\": 0.93125,\n        \"num_unique_values\": 2,\n        \"samples\": [\n          0.93125\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  },\n  \"type\": \"dataframe\", \"variable_name\": \"tuning_result\"}

comparison = pd.DataFrame({
    "Criterion": [
        "Accuracy",
        "Model Complexity",
        "Training Time",
        "Interpretability"
    ],
    "Logistic Regression": [
        accuracy_score(y_test, logistic_pred),
        "Low",
        logistic_time,
        "High"
    ],

```

```

    "SVM": [
        accuracy_score(y_test, best_prediction),
        "High",
        min(linear_time, poly_time, rbf_time, sigmoid_time),
        "Low"
    ]
})

comparison

{"summary": "{\n  \"name\": \"comparison\", \n  \"rows\": 4, \n  \"fields\": [\n    {\n      \"column\": \"Criterion\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 4, \n        \"samples\": [\n          \"Model Complexity\", \n          \"Interpretability\", \n          \"Accuracy\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"Logistic Regression\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 4, \n        \"samples\": [\n          \"Low\", \n          \"High\", \n          0.9196525515743756 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"SVM\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 4, \n        \"samples\": [\n          \"High\", \n          \"Low\", \n          0.9348534201954397 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    ] \n  }, \n  \"type\": \"dataframe\", \n  \"variable_name\": \"comparison\"} models\n= [\"Logistic Regression\", \"Best SVM\"]

scores = [
    accuracy_score(y_test, logistic_pred),
    accuracy_score(y_test, best_prediction) ]

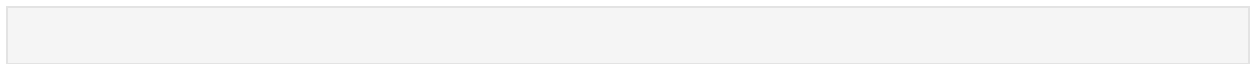
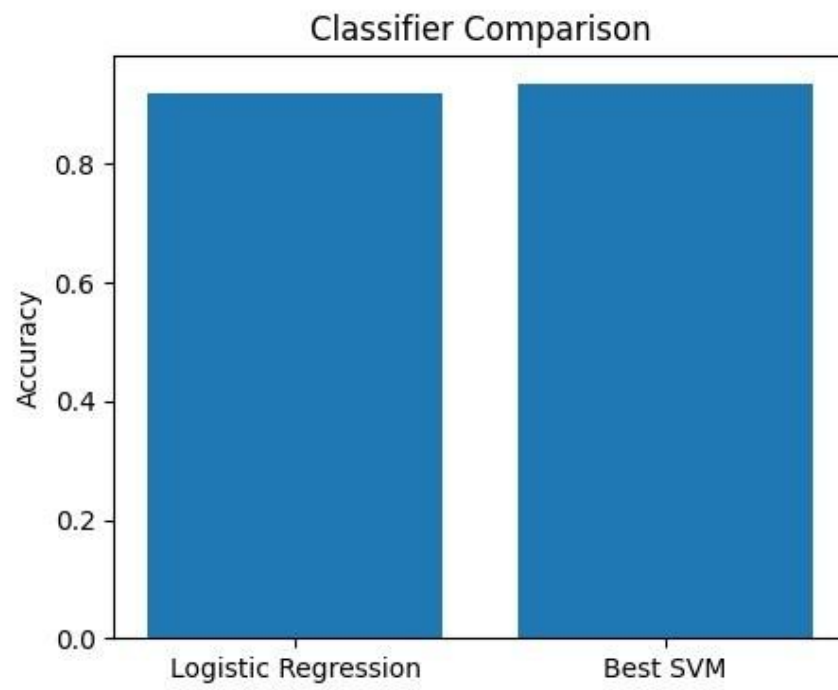
plt.figure(figsize=(5,4))

plt.bar(models, scores)

plt.ylabel("Accuracy")

plt.title("Classifier Comparison") plt.show()

```



Visualizations

The following plots were generated.

- Spam and Ham class distribution
- Correlation heatmap
- Confusion matrix for Logistic Regression
- Confusion matrix for the best SVM model
- Classifier comparison bar chart

Performance Tables

The models were evaluated using the following metrics.

- Accuracy
- Precision
- Recall
- F1 Score
- Training Time

The hyperparameter tuning results, cross-validation results, kernel comparison and performance tables are included in the attached notebook.

Comparative Analysis

Logistic Regression and SVM were compared based on their accuracy, training time and model behaviour. The effect of different SVM kernels and hyperparameter tuning was also observed.

Learning Outcomes

- Learned how Logistic Regression is used for binary classification.
- Understood the working of Support Vector Machine.
- Performed hyperparameter tuning using GridSearchCV.
- Evaluated the models using standard performance metrics.
- Compared the performance of Logistic Regression and SVM.

Conclusion

In this experiment, Logistic Regression and Support Vector Machine were used for email spam classification. Both models were evaluated using different performance metrics. Hyperparameter tuning and cross-validation helped improve the model performance and made it easier to compare the two classifiers.

